Data Cleaning and Data Preprocessing:

1. Data cleaning is the process of changing or eliminating garbage, incorrect, duplicate, corrupted, or incomplete data in a dataset.
2. There's no such absolute way to describe the precise steps in the data cleaning process because the processes may vary from dataset to dataset.

Data Cleaning Cycle

Missing Values:

```
# import the pandas library
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f',
'h'],columns=['one', 'two', 'three'])
print(df)
df = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])

print (df)
```

```
print (df)
```

```
             one       two     three
     a -0.312295  0.334567  1.256013
     c -0.310979 -0.610969 -0.248298
     e -0.457244  0.667382 -1.047236
     f  0.068153  1.879339  0.579019
     h -0.351475  0.830226  0.668234
             one       two     three
     a -0.312295  0.334567  1.256013
     b       NaN       NaN       NaN
     c -0.310979 -0.610969 -0.248298
     d       NaN       NaN       NaN
     e -0.457244  0.667382 -1.047236
     f  0.068153  1.879339  0.579019
     g       NaN       NaN       NaN
     h -0.351475  0.830226  0.668234
```

Check for Missing Values:

To make detecting missing values easier (and across different array dtypes),Pandas provides the **isnull()** and **notnull()** functions, which are also methods on Series and DataFrame objects –

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f',
'h'],columns=['one', 'two', 'three'])

df = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])

print (df['one'].isnull())
```

```
     a    False
     b     True
     c    False
     d     True
     e    False
     f    False
     g     True
     h    False
     Name: one, dtype: bool
```

Replacing the Missing Values

```
#Replace the missing values by 0
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randn(3, 3), index=['a', 'c', 'e'],columns=['one',
'two', 'three'])
df = df.reindex(['a', 'b', 'c'])
print (df)
print ("NaN replaced with '0':")
print (df.fillna(0))
```

```
             one       two     three
     a  1.754148 -0.496140  1.098183
     b       NaN       NaN       NaN
     c  2.523064  0.401495  2.000023
     NaN replaced with '0':
             one       two     three
     a  1.754148 -0.496140  1.098183
     b  0.000000  0.000000  0.000000
     c  2.523064  0.401495  2.000023
```

Fill NA Forward and Backward

```
# Method           Action
pad/fill          Fill methods Forward
bfill/backfill    Fill methods Backward
```

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f',
'h'],columns=['one', 'two', 'three'])
df = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])
print(df)
print (df.fillna(method='pad'))
```

```
             one       two     three
     a -0.226089  0.111366 -0.679062
     b       NaN       NaN       NaN
     c  1.398571 -0.533240  1.434607
     d       NaN       NaN       NaN
     e -0.670727 -0.118627  1.413989
     f -0.737852  0.488276 -0.213893
     g       NaN       NaN       NaN
     h -1.811222  2.075223 -0.270869
             one       two     three
     a -0.226089  0.111366 -0.679062
     b -0.226089  0.111366 -0.679062
     c  1.398571 -0.533240  1.434607
     d  1.398571 -0.533240  1.434607
     e -0.670727 -0.118627  1.413989
     f -0.737852  0.488276 -0.213893
     g -0.737852  0.488276 -0.213893
     h -1.811222  2.075223 -0.270869
```

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f',
'h'],columns=['one', 'two', 'three'])
df = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])

print (df.fillna(method='bfill'))
```

```
             one       two     three
     a  0.176095 -0.524348 -0.271432
     b  0.280205 -0.866840 -0.646124
     c  0.280205 -0.866840 -0.646124
     d -1.252666  0.025970 -0.274940
     e -1.252666  0.025970 -0.274940
     f -2.487715 -0.552433  1.150178
     g -0.098895  1.262921  1.934049
     h -0.098895  1.262921  1.934049
```

Drop Missing Values:

Use dropna function along with the axis argument.

By default, axis=0, i.e., along row, which means that if any value within a row is NA then the whole row is excluded.

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randn(5, 3), index=['a', 'c', 'e', 'f',
'h'],columns=['one', 'two', 'three'])
#print(df)
df = df.reindex(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])
print(df)
print (df.dropna())
```

```
             one       two     three
     a -1.009441 -1.047061  0.326038
     b       NaN       NaN       NaN
     c -1.190279  1.478310  0.342631
     d       NaN       NaN       NaN
     e  0.731494 -0.018962  0.066996
     f  0.401238  0.382164 -1.136754
     g       NaN       NaN       NaN
     h  0.988150 -1.064995  0.466845
             one       two     three
     a -1.009441 -1.047061  0.326038
     c -1.190279  1.478310  0.342631
     e  0.731494 -0.018962  0.066996
     f  0.401238  0.382164 -1.136754
     h  0.988150 -1.064995  0.466845
```

Replace Missing (or) Generic Values:

We can achieve this by applying the **replace** method.

Replacing NA with a scalar value is equivalent behavior of the **fillna()** function.

```
import pandas as pd
import numpy as np
df = pd.DataFrame({'one':[10,20,30,40,50,2000],
'two':[1000,0,30,40,50,60]})
print(df)
print (df.replace({1000:10,2000:60}))
```

```
     one   two
0    10  1000
1    20     0
2    30    30
3    40    40
4    50    50
5  2000    60
   one  two
0   10   10
1   20    0
2   30   30
3   40   40
4   50   50
5   60   60
```

Data Preprocessing

1. Load data in Pandas
2. Drop columns that aren't useful
3. Drop rows with missing values
4. Create dummy variables
5. Take care of missing data
6. Convert the data frame to NumPy

Download Titanic-Dataset from Kaggle.com.
Here we are going to use train.csv dataset for preprocessing.

```
import pandas as pd
import numpy as np
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
df = pd.read_csv("/content/drive/My Drive/Data/train.csv")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

Drop the Columns that are not required

```
cols=['Name','Ticket','Cabin']
df=df.drop(cols, axis=1)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
```

```
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Sex          891 non-null    object
 4   Age          714 non-null    float64
 5   SibSp        891 non-null    int64
 6   Parch        891 non-null    int64
 7   Fare         891 non-null    float64
 8   Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(2)
memory usage: 62.8+ KB
```

Drop the rows having no values

```
df = df.dropna()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 712 entries, 0 to 890
Data columns (total 9 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  712 non-null    int64
 1   Survived     712 non-null    int64
 2   Pclass       712 non-null    int64
 3   Sex          712 non-null    object
 4   Age          712 non-null    float64
 5   SibSp        712 non-null    int64
 6   Parch        712 non-null    int64
 7   Fare         712 non-null    float64
 8   Embarked     712 non-null    object
dtypes: float64(2), int64(5), object(2)
memory usage: 55.6+ KB
```

Creating Dummy variables

Instead of wasting our data, let's convert the Pclass, Sex and Embarked to columns in Pandas and drop them after conversion.

```
dummies = []
cols = ['Pclass', 'Sex', 'Embarked']
for col in cols:
   dummies.append(pd.get_dummies(df[col]))
```

Transfor the eigth columns

```
titanic_dummies = pd.concat(dummies, axis=1)
```

Concatenate the values with data frame

```
df = pd.concat((df,titanic_dummies), axis=1)
```

Remove the unwanted cols

```
df = df.drop(['Pclass', 'Sex', 'Embarked'], axis=1)
```

Take care of Missing data

Let's compute a **median or interpolate()** all the ages and fill those missing age values. Pandas has an interpolate() function that will replace all the missing NaNs to interpolated values.

```
df['Age'] = df['Age'].interpolate()
print(df)
```

Min Max Scaler and Standardization

Normalization is a rescaling of the data from the original range so that all values are within the new range of 0 and 1.

A value is normalized as follows:

y = (x – min) / (max – min)

```
from sklearn.preprocessing import MinMaxScaler
data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
scaler = MinMaxScaler()
print(scaler.fit(data))
MinMaxScaler()
print(scaler.data_max_)
print(scaler.transform(data))
```

```
MinMaxScaler()
[ 1. 18.]
[[0.   0.  ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
```

Data Standardization

Standardizing a dataset involves rescaling the distribution of values so that the mean of observed values is 0 and the standard deviation is 1.

A value is standardized as follows:

y = (x – mean) / standard_deviation

Where the mean is calculated as:

mean = sum(x) / count(x)

And the standard_deviation is calculated as:

standard_deviation = sqrt(sum((x – mean)^2) / count(x))

```
from numpy import asarray
from sklearn.preprocessing import StandardScaler
# define data
data = asarray([[100, 0.001],
 [8, 0.05],
 [50, 0.005],
 [88, 0.07],
 [4, 0.1]])
print(data)
# define standard scaler
scaler = StandardScaler()
# transform data
scaled = scaler.fit_transform(data)
print(scaled)
```

```
[[1.0e+02 1.0e-03]
 [8.0e+00 5.0e-02]
 [5.0e+01 5.0e-03]
 [8.8e+01 7.0e-02]
 [4.0e+00 1.0e-01]]
[[ 1.26398112 -1.16389967]
 [-1.06174414  0.12639634]
 [ 0.         -1.05856939]
 [ 0.96062565  0.65304778]
 [-1.16286263  1.44302493]]
```